

INF560 / Unidad 3 / GL05

GUÍA DE LABORATORIO

Base de datos y migraciones con PostgreSQL

Del arreglo estático a la persistencia real en eventos-app

Laravel 13 · PostgreSQL 18 · Proyecto acumulativo eventos-app · Duración 3 h

Métodos HTTP de la serie: **GET** **POST** **PUT** **DELETE**

Universidad Autónoma Tomás Frías · Facultad de Ciencias Puras · Ingeniería Informática

00 Antes de empezar

Esta guía continúa el proyecto acumulativo [eventos-app](#). En la GL04 usted construyó las vistas con Blade y Tailwind, y la lista de eventos se alimentaba de un **arreglo estático** definido dentro del controlador. Ese arreglo cumplió su propósito para practicar la capa de presentación, pero tiene limitaciones que ahora vamos a resolver conectando la aplicación a una base de datos real.

¿POR QUÉ ABANDONAR EL ARREGLO ESTÁTICO?

- No persiste: los datos viven en el código; no puede crear, editar ni borrar eventos desde la aplicación.
- No escala: cargar cientos de eventos en un arreglo es inmanejable y lento.
- No se comparte: cada estudiante tendría datos distintos codificados a mano en su repositorio.

Objetivos de la guía

- Instalar y preparar PostgreSQL 18, crear la base de datos y un usuario dedicado con sus permisos.
- Configurar la conexión de Laravel 13 a PostgreSQL a través del archivo `.env`.
- Comprender qué es una migración y por qué versiona el esquema de la base de datos.
- Diseñar y ejecutar la migración de la tabla `eventos`.
- Evolucionar el esquema con una migración adicional, sin destruir lo ya creado.

Requisitos previos

- El proyecto [eventos-app](#) de la GL04 funcionando (Laravel 13 + Tailwind v4).
- PHP 8.4, Composer y Git ya instalados (GL01).
- VS Code con la terminal integrada abierta en la carpeta del proyecto.

ALCANCE DE ESTA GUÍA

Aquí llegamos hasta tener la tabla `eventos` creada y verificada en PostgreSQL. El **modelo Evento** y las consultas con **Eloquent** —es decir, reemplazar el arreglo por la base de datos en el controlador— son el tema de la **GL06**. Por eso al final de esta guía insertaremos un registro de prueba "a mano" solo para comprobar que la tabla existe y responde.

01 Teoría breve: bases de datos, PostgreSQL y migraciones

1.1 Base de datos relacional

Una **base de datos relacional** organiza la información en **tablas**. Cada tabla se parece a una hoja de cálculo: tiene **columnas** (los campos, con un tipo de dato fijo: texto, número, fecha...) y **filas** (cada registro concreto). Cada fila se identifica por una **llave primaria** única, normalmente una columna `id` que se incrementa sola. La diferencia con una hoja de cálculo es que la base de datos **hace cumplir reglas**: tipos de dato, obligatoriedad, unicidad y relaciones entre tablas.

1.2 ¿Por qué PostgreSQL?

Usaremos **PostgreSQL** durante toda la serie. Es un gestor relacional **libre, maduro y muy usado en la industria**. Respeta estrictamente el estándar SQL, garantiza transacciones seguras (ACID), y ofrece tipos avanzados (JSON, arreglos, búsqueda de texto) que aprovecharemos más adelante. Laravel lo soporta oficialmente, así que la integración es directa.

1.3 ¿Cómo se conecta Laravel a la base de datos?

Laravel no guarda credenciales en el código. Lee la configuración desde el archivo `.env` (variables de entorno), que a su vez alimenta a `config/database.php`. Por debajo, Laravel usa **PDO**, la capa de PHP para hablar con bases de datos, y para PostgreSQL necesita la extensión `pdo_pgsql` habilitada en PHP. Si esa extensión falta, verá el error `could not find driver`.

1.4 ¿Qué es una migración?

DEFINICIÓN — MIGRACIÓN

Una **migración** es un archivo PHP que describe, en código, un cambio en el esquema de la base de datos (crear una tabla, agregar una columna, etc.). Es el **control de versiones del esquema**: vive en Git junto al proyecto, se ejecuta con un comando y produce siempre el mismo resultado en cualquier máquina.

Cada migración tiene dos métodos: `up()`, que aplica el cambio, y `down()`, que lo revierte. Gracias a esto, en lugar de crear tablas a mano en pgAdmin —algo que nadie más podría reproducir—, todo el equipo comparte el mismo esquema simplemente ejecutando las migraciones. Este es el flujo profesional y el que usaremos siempre.

02 Ejercicio 1 — Preparar PostgreSQL

El objetivo de este ejercicio es dejar PostgreSQL listo, con la base de datos del proyecto y un usuario dedicado creados, antes de tocar Laravel.

Paso 1 — Verificar que PostgreSQL y el driver estén instalados

En la terminal, confirme la versión de PostgreSQL y que PHP tenga habilitada la extensión `pdo_pgsql`:

```
terminal Copiar
$ psql --version
psql (PostgreSQL) 18.0

$ php -m | grep pgsql
pdo_pgsql
pgsql
```

NOTA

Si `php -m | grep pgsql` no imprime nada, la extensión está deshabilitada. Abra su `php.ini` y quite el `;` delante de `extension=pdo_pgsql`; luego reinicie la terminal. Encuentra la ruta de su `php.ini` con `php --ini`.

Paso 2 — Crear un usuario dedicado y la base de datos

En lugar de conectar la aplicación con el superusuario `postgres` (que puede hacer todo en el servidor), crearemos un **usuario dedicado** con permisos solo sobre esta base de datos. Es la práctica recomendada: si esas credenciales se filtran, el daño queda acotado a `eventos_app`.

Ingresa a la consola como superusuario y ejecute estas tres instrucciones SQL:

```
psql -U postgres Copiar
-- 1. Crear el usuario (rol) con su contraseña
CREATE USER eventos_user WITH PASSWORD 'clave_segura';
```

```
-- 2. Crear la base y hacer que ese usuario sea su dueño
CREATE DATABASE eventos_app OWNER eventos_user;

-- 3. Concederle todos los privilegios sobre la base
GRANT ALL PRIVILEGES ON DATABASE eventos_app TO eventos_user;
```

Cada instrucción responde con una confirmación (`CREATE ROLE`, `CREATE DATABASE`, `GRANT`). Reemplace `'clave_segura'` por una contraseña real y anótela: irá en el `.env` del proyecto.

¿POR QUÉ OWNER Y NO SOLO GRANT?

Desde PostgreSQL 15, un usuario común no puede crear tablas en el esquema `public` solo con `GRANT ... ON DATABASE`. Al hacer a `eventos_user` **dueño** de la base (`OWNER`), obtiene control del esquema `public` y podrá crear las tablas al ejecutar las migraciones. Si la base ya existía a nombre de `postgres`, concédale el esquema con `GRANT ALL ON SCHEMA public TO eventos_user;`.

Paso 3 — Conectarse y explorar con `psql`

Conéctese ahora **con el usuario dedicado** a la base recién creada:

```
terminal                                                                    Copiar

$ psql -U eventos_user -d eventos_app
Password for user eventos_user: *****
psql (18.0)
Type "help" for help.

eventos_app=> \conninfo
You are connected to database "eventos_app" as user "eventos_user"
via socket in "/var/run/postgresql" at port "5432".
```

El prompt es `eventos_app=>`; el `>` indica un usuario común (un superusuario mostraría `#`). Estos son los meta-comandos de `psql` para navegar y explorar la base, que usará durante todo el semestre:

Comando	Qué muestra
<code>\l</code>	Lista todas las bases de datos del servidor.
<code>\c eventos_app</code>	Se conecta (cambia) a la base <code>eventos_app</code> .
<code>\conninfo</code>	A qué base y como qué usuario está conectado.
<code>\dt</code>	Lista las tablas de la base actual.
<code>\dt+</code>	Tablas con su tamaño en disco.
<code>\d eventos</code>	Describe una tabla: columnas, tipos y restricciones.
<code>\d+ eventos</code>	Descripción extendida (índices y comentarios).
<code>\dn</code>	Lista los esquemas.
<code>\du</code>	Lista los usuarios/roles y sus atributos.
<code>\di</code>	Lista los índices.
<code>\x</code>	Alterna la vista expandida (una columna por línea).
<code>\?</code>	Ayuda de todos los meta-comandos.
<code>\q</code>	Salida de <code>psql</code> .

Comando Qué muestra

NOTA

Si ejecuta `\dt` ahora verá `Did not find any relations`: la base está vacía. Eso cambia en el Ejercicio 4, cuando las migraciones creen la tabla `eventos`. Vuelva a `\dt` después de migrar para verla aparecer.

03 Ejercicio 2 — Configurar la conexión en Laravel

El proyecto se creó en la GL03 usando `SQLite` (un archivo local, cómodo para la fase de frontend). Ahora lo cambiaremos a PostgreSQL editando el archivo `.env` en la raíz del proyecto.

Paso 1 — Editar el archivo `.env`

Busque el bloque de conexión y déjelo así (use **sus** credenciales reales de PostgreSQL):

```
.env
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=eventos_app
DB_USERNAME=eventos_user
DB_PASSWORD=clave_segura
```

[Copiar](#)

Cada variable significa:

Variable	Significado
DB_CONNECTION	Driver a usar. <code>pgsql</code> = PostgreSQL.
DB_HOST	Dónde corre el servidor. <code>127.0.0.1</code> = su propia máquina.
DB_PORT	Puerto de PostgreSQL. Por defecto <code>5432</code> .
DB_DATABASE	Nombre de la base de datos creada en el Ejercicio 1.
DB_USERNAME	Usuario dedicado creado en el Ejercicio 1 (<code>eventos_user</code>).
DB_PASSWORD	La contraseña de ese usuario.

NOTA

Si en su `.env` había líneas de `SQLite` comentadas o la variable `DB_DATABASE` apuntaba a un archivo `.sqlite`, reemplácelas por las de arriba. El archivo `.env` nunca se sube a GitHub (está en `.gitignore`): guarda secretos.

Paso 2 — Limpiar la configuración en caché y verificar la conexión

Laravel puede tener la configuración cacheada. Límpiela y luego pida a Laravel que describa la conexión con `db:show`:

```
terminal
$ php artisan config:clear
INFO Configuration cache cleared successfully.

$ php artisan db:show
PostgreSQL 18.0 .....
Connection ..... pgsql
```

[Copiar](#)

```
Database ..... eventos_app
Host ..... 127.0.0.1
Tables ..... 0
```

Ver **Connection: pgsql** y **Database: eventos_app** confirma que Laravel ya habla con PostgreSQL. Que haya **0 Tables** es normal: todavía no hemos migrado nada.

ERRORES DE CONEXIÓN MÁS COMUNES

- **could not find driver** → falta la extensión **pdo_pgsql** (ver Ejercicio 1, Paso 1).
- **password authentication failed** → usuario o contraseña incorrectos en **.env**.
- **database "eventos_app" does not exist** → no creó la base de datos, o la escribió distinta.
- **Connection refused** → el servicio de PostgreSQL no está corriendo; inícielo desde los servicios del sistema.

04 Ejercicio 3 — Anatomía de una migración

Laravel viene con algunas migraciones por defecto (para usuarios, caché y colas de trabajo). Vamos a leerlas para entender la estructura, y luego ejecutarlas. Abra la carpeta **database/migrations** en VS Code y abra el archivo que crea la tabla **users**.

Verá una estructura como esta (simplificada):

```
database/migrations/0001_01_01_000000_create_users_table.php Copiar

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    // Se aplica cuando ejecuta: php artisan migrate
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id(); // llave primaria autoincremental
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps(); // created_at y updated_at
        });
    }

    // Se aplica cuando revierte: php artisan migrate:rollback
    public function down(): void
    {
        Schema::dropIfExists('users');
    }
};
```

Puntos clave para leer cualquier migración:

- **up()** define el cambio; **down()** lo deshace (aquí, borra la tabla).

- `Schema::create()` crea una tabla nueva; recibe el nombre y una función que recibe un `Blueprint $table`.
- Sobre `$table` se declaran las columnas: `$table->string('name')` crea una columna de texto llamada `name`.
- `$table->id()` crea la llave primaria; `$table->timestamps()` crea `created_at` y `updated_at` automáticamente.

Ejecutar las migraciones

```
terminal Copiar
$ php artisan migrate

INFO  Running migrations.

0001_01_01_000000_create_users_table ..... DONE
0001_01_01_000001_create_cache_table ..... DONE
0001_01_01_000002_create_jobs_table ..... DONE
```

Revise el estado con `migrate:status`:

```
terminal Copiar
$ php artisan migrate:status
Migration name ..... Batch / Status
0001_01_01_000000_create_users_table ..... [1] Ran
0001_01_01_000001_create_cache_table ..... [1] Ran
0001_01_01_000002_create_jobs_table ..... [1] Ran
```

¿CÓMO SABE LARAVEL QUÉ YA SE EJECUTÓ?

Al migrar por primera vez, Laravel crea una tabla especial llamada `migrations` donde anota cada archivo ejecutado y su número de **lote** (`batch`). Así, si vuelve a correr `migrate`, solo aplica las migraciones nuevas, nunca repite las anteriores.

Comandos de migración que usará todo el semestre

Comando	Qué hace
<code>php artisan migrate</code>	Aplica las migraciones pendientes.
<code>php artisan migrate:status</code>	Lista cuáles se ejecutaron y cuáles no.
<code>php artisan migrate:rollback</code>	Revierde el último lote (llama a <code>down()</code>).
<code>php artisan migrate:refresh</code>	Revierde todo y vuelve a migrar.
<code>php artisan migrate:fresh</code>	Borra todas las tablas y migra desde cero.

CUIDADO CON FRESH Y REFRESH

Ambos **borran los datos**. En desarrollo son útiles para empezar limpio, pero **nunca** se ejecutan sobre una base de datos con información real que quiera conservar.

05 Ejercicio 4 — Crear la tabla eventos

Ahora crearemos la tabla que dará vida a `eventos-app`. Diseñaremos sus columnas a partir de los mismos datos que ya mostraba el arreglo de la GL04.

Paso 1 — Generar el archivo de migración

```
terminal Copiar
$ php artisan make:migration create_eventos_table

INFO Migration created successfully.
```

Esto crea un archivo con la fecha y hora en `database/migrations/`. Al nombrarlo `create_eventos_table`, Laravel ya prepara un `Schema::create('eventos', ...)` listo para completar.

Paso 2 — Diseñar el esquema

Antes de escribir código, decidimos las columnas y por qué elegimos cada tipo:

Columna	Tipo	Por qué
titulo	string	Nombre del evento (texto corto).
slug	string único	Identificador para la URL; único para evitar duplicados.
descripcion	text	Texto largo, sin límite práctico de caracteres.
fecha	dateTime	Fecha y hora en que ocurre el evento.
lugar	string	Nombre del recinto o dirección.
cupos	unsignedInteger	Número de asistentes; nunca negativo.
precio	decimal(8,2)	Dinero: 2 decimales exactos (no float).
imagen	string nullable	Ruta de la portada; opcional.

DOS DECISIONES IMPORTANTES DE TIPO

- **Dinero con decimal, no float.** Los `float` tienen errores de redondeo; `decimal(8,2)` guarda hasta 999999.99 con precisión exacta.
- **Opcional con nullable.** `imagen` puede quedar vacío al inicio, así que se marca `->nullable()`. Las demás columnas son obligatorias.

Paso 3 — Escribir la migración

Abra la migración recién creada y complete el método `up()` así:

```
database/migrations/XXXX_create_eventos_table.php Copiar

public function up(): void
{
    Schema::create('eventos', function (Blueprint $table) {
        $table->id();
        $table->string('titulo');
        $table->string('slug')->unique();
        $table->text('descripcion');
        $table->dateTime('fecha');
        $table->string('lugar');
        $table->unsignedInteger('cupos');
        $table->decimal('precio', 8, 2)->default(0);
        $table->string('imagen')->nullable();
        $table->timestamps();
    });
}

public function down(): void
{
}
```



```
Schema::dropIfExists('eventos');
}
```

Note `->default(0)` en `precio`: un evento sin precio asignado se considera gratuito. El `down()` simplemente borra la tabla, para poder revertir de forma limpia.

Paso 4 — Ejecutar y verificar

terminal

Copiar

```
$ php artisan migrate

XXXX_XX_XX_XXXXXX_create_eventos_table ..... DONE
```

Confirme la estructura real en PostgreSQL con `psql` y el comando `\d eventos`:

terminal

Copiar

```
$ psql -U eventos_user -d eventos_app
eventos_app=> \d eventos

          Table "public.eventos"
   Column   |      Type      | Nullable | Default
-----+-----+-----+-----
 id          | bigint          | not null | nextval(...)
 titulo      | varchar(255)    | not null |
 slug        | varchar(255)    | not null |
 descripcion | text            | not null |
 fecha       | timestamp       | not null |
 lugar       | varchar(255)    | not null |
 cupo        | integer         | not null |
 precio      | numeric(8,2)    | not null | 0
 imagen      | varchar(255)    |          |
 created_at  | timestamp       |          |
 updated_at  | timestamp       |          |
```

Paso 5 — Insertar un registro de prueba (sin modelo aún)

Para comprobar que la tabla funciona, insertaremos una fila con `php artisan tinker` usando la fachada `DB` (SQL directo). Todavía **no** usamos un modelo Eloquent: eso llega en la GL06.

php artisan tinker

Copiar

```
> DB::table('eventos')->insert([
    'titulo'      => 'Concierto de aniversario UATF',
    'slug'        => 'concierto-aniversario-uatf',
    'descripcion' => 'Noche de música en vivo en el paraninfo.',
    'fecha'       => '2026-11-05 20:00:00',
    'lugar'       => 'Paraninfo Universitario',
    'cupo'        => 300,
    'precio'      => 25.00,
    'created_at'  => now(),
    'updated_at'  => now(),
]);
= true

> DB::table('eventos')->count();
= 1
```

LO QUE ACABA DE LOGRAR

La tabla `eventos` existe en PostgreSQL y ya guarda datos de verdad. En la **GL06** crearemos el modelo `Evento` y reemplazaremos el arreglo estático del controlador por consultas Eloquent a esta tabla, para que el CRUD de la aplicación funcione de principio a fin.

06 Ejercicio 5 — Evolucionar el esquema

En un proyecto real el esquema cambia con el tiempo. Supongamos que ahora necesitamos marcar si un evento está **publicado** o sigue en borrador. Agregaremos una columna `publicado` — pero con **una migración nueva**, no editando la anterior.

REGLA DE ORO

Nunca edite una migración que ya se ejecutó y que sus compañeros o el servidor ya aplicaron. Sus máquinas no volverían a correrla y quedarían con esquemas distintos. Todo cambio posterior va en **una migración nueva**.

Paso 1 — Generar la migración de modificación

```
terminal Copiar
$ php artisan make:migration add_publicado_to_eventos_table
```

Paso 2 — Usar `Schema::table` (modificar, no crear)

```
database/migrations/XXXX_add_publicado_to_eventos_table.php Copiar

public function up(): void
{
    Schema::table('eventos', function (Blueprint $table) {
        $table->boolean('publicado')->default(false)->after('precio');
    });
}

public function down(): void
{
    Schema::table('eventos', function (Blueprint $table) {
        $table->dropColumn('publicado');
    });
}
```

Aquí usamos `Schema::table` (no `create`) porque la tabla ya existe. `->default(false)` da un valor a las filas existentes; `->after('precio')` coloca la columna en esa posición. El `down()` quita la columna si se revierte.

Paso 3 — Migrar y confirmar

```
terminal Copiar

$ php artisan migrate

XXXX_XX_XX_XXXXXX_add_publicado_to_eventos_table ... DONE

$ php artisan migrate:status # la nueva aparece en un batch mayor
```

NOTA

Fíjese en que la nueva migración obtiene un número de `batch` superior. Si ejecutara `migrate:rollback`, solo se revertiría este último cambio (quitar `publicado`), no toda la tabla.

07 Trabajo independiente

Aplice lo aprendido creando, por su cuenta, la tabla `categorias`, que servirá para clasificar los eventos (conciertos, conferencias, deportes...). La usaremos al estudiar relaciones en la GL09.

Consigna

- Cree una migración `create_categorias_table`.
- La tabla debe tener: `id`, `nombre` (string), `slug` (string único), `descripcion` (text, nullable) y `timestamps`.
- Ejecute la migración y verifique la estructura con `\d categorias`.
- Inserte 3 categorías de prueba con `Tinker` usando la fachada `DB`.

Entregables

- Commit atómico con un mensaje claro, por ejemplo: `git commit -m "feat: migración de tabla categorias"`.
- Push a su repositorio `INF560-2026-02-Apellido`.
- Captura de pantalla de la estructura de la tabla y del conteo de registros.

08 Checklist de verificación

- ☐ PostgreSQL 18 responde y la extensión `pdo_pgsql` está habilitada.
- ☐ La base de datos `eventos_app` existe.
- ☐ `.env` apunta a `pgsql` con las credenciales correctas y `php artisan db:show` lo confirma.
- ☐ Las migraciones por defecto se ejecutaron (`users`, `cache`, `jobs`).
- ☐ La tabla `eventos` existe con todas sus columnas y tipos correctos.
- ☐ Insertó y contó al menos un registro de prueba con `Tinker`.
- ☐ Agregó la columna `publicado` mediante una migración nueva.
- ☐ Creó la tabla `categorias` (trabajo independiente) y la subió a GitHub.

09 Problemas frecuentes

Síntoma	Causa y solución
<code>could not find driver</code>	La extensión <code>pdo_pgsql</code> no está activa. Habilítela en <code>php.ini</code> y reinicie la terminal.
<code>password authentication failed</code>	Usuario o contraseña incorrectos en <code>.env</code> . Verifique <code>DB_USERNAME</code> y <code>DB_PASSWORD</code> .
<code>database "... " does not exist</code>	No creó la base o el nombre no coincide con <code>DB_DATABASE</code> . Créela con <code>CREATE DATABASE</code> .
<code>role "... " does not exist</code>	El usuario dedicado no se creó o no coincide con <code>DB_USERNAME</code> . Créelo con <code>CREATE USER eventos_user WITH PASSWORD '...'</code> .

Síntoma	Causa y solución
permission denied for schema public	El usuario no es dueño de la base. Recréela con OWNER, o ejecute GRANT ALL ON SCHEMA public TO eventos_user;.
Connection refused	El servicio de PostgreSQL está detenido. Inícielo desde los servicios del sistema.
Los cambios de .env no surten efecto	Configuración cacheada. Ejecute php artisan config:clear.
Nothing to migrate	Ya estaban aplicadas. Use migrate:status para revisar; migrate:fresh para reiniciar (borra datos).

10 Comandos de referencia

comandos usados en esta guía		Copiar
<pre># PostgreSQL – crear usuario y base (como superusuario) psql -U postgres CREATE USER eventos_user WITH PASSWORD 'clave_segura'; CREATE DATABASE eventos_app OWNER eventos_user; GRANT ALL PRIVILEGES ON DATABASE eventos_app TO eventos_user; # PostgreSQL – navegar y explorar psql -U eventos_user -d eventos_app # entrar a la base \conninfo # base y usuario actuales \l # listar bases \dt # listar tablas \d eventos # describir una tabla \du # listar usuarios/roles \x # vista expandida \q # salir # Laravel – conexión php artisan config:clear # limpiar config en caché php artisan db:show # ver la conexión activa # Laravel – migraciones php artisan make:migration create_eventos_table php artisan migrate # aplicar pendientes php artisan migrate:status # ver estado php artisan migrate:rollback # revertir último lote php artisan migrate:fresh # borrar todo y migrar (¡borra datos!)</pre>		

11 Rúbrica de evaluación

Criterio	Pts
PostgreSQL preparado y base de datos eventos_app creada	15
Conexión de Laravel a PostgreSQL configurada y verificada	20
Migración de la tabla eventos con tipos correctos y ejecutada	25
Registro de prueba insertado y verificado con Tinker	10
Migración adicional (columna publicado) bien planteada	15
Trabajo independiente: tabla categorias en GitHub	10

Criterio	Pts
Buenas prácticas de Git (commits atómicos y claros)	5
Total	100